```
/* Design: Chrome & Asphalt — isometric 3D floor plan */
import { motion } from "framer-motion";
import { useInView } from "framer-motion";
import { useRef, useState } from "react";

const zones = [
  {
    id: 1,
    label: "Мотошкола",
    icon: "🏍️",
    color: "#FF4500",
    desc: "Обучение с нуля, безопасная площадка, опытные инструкторы",
    // isometric grid position (col, row) and size
    col: 0, row: 1, cols: 2, rows: 2,
  },
  {
    id: 2,
    label: "Дрифт-картинг",
    icon: "🏁",
    color: "#E74C3C",
    desc: "Адреналин контролируемого заноса на закрытой трассе",
    col: 0, row: 3, cols: 2, rows: 2,
  },
  {
    id: 3,
    label: "Детейлинг",
    icon: "✨",
    color: "#27AE60",
    desc: "Профессиональный уход за автомобилем",
    col: 3, row: 0, cols: 2, rows: 2,
  },
  {
    id: 4,
    label: "Мото-Салон",
    icon: "🏍️",
    color: "#2980B9",
    desc: "Шоурум мотоциклов и экипировки",
    col: 5, row: 0, cols: 2, rows: 2,
  },
  {
    id: 5,
    label: "Музей JDM",
    icon: "🇯🇵",
    color: "#9B59B6",
```

```
    desc: "Культовые японские авто и атмосфера JDM культуры",
    col: 3, row: 3, cols: 2, rows: 2,
  },
];

// Isometric projection constants
const TILE_W = 64;
const TILE_H = 32;
const WALL_H = 28;

function toIso(col: number, row: number) {
  return {
    x: (col - row) * (TILE_W / 2),
    y: (col + row) * (TILE_H / 2),
  };
}

function IsoBlock({
  col, row, cols, rows, color, label, icon, isActive, onEnter, onLeave, delay,
}: {
  col: number; row: number; cols: number; rows: number;
  color: string; label: string; icon: string;
  isActive: boolean; onEnter: () => void; onLeave: () => void; delay: number;
}) {
  const tl = toIso(col, row);
  const tr = toIso(col + cols, row);
  const br = toIso(col + cols, row + rows);
  const bl = toIso(col, row + rows);

  const topFace = `${tl.x},${tl.y} ${tr.x},${tr.y} ${br.x},${br.y} ${bl.x},${bl.y

  // Left face (bottom-left side)
  const leftFace = `${bl.x},${bl.y} ${bl.x},${bl.y + WALL_H} ${tl.x},${tl.y + WAL
  // Right face (bottom-right side)
  const rightFace = `${br.x},${br.y} ${br.x},${br.y + WALL_H} ${bl.x},${bl.y + WA

  const cx = (tl.x + tr.x + br.x + bl.x) / 4;
  const cy = (tl.y + tr.y + br.y + bl.y) / 4;

  const alpha = isActive ? "cc" : "33";
  const wallAlpha = isActive ? "99" : "22";

  return (
    <motion.g
      initial={{ opacity: 0, y: -20 }}
      animate={{ opacity: 1, y: 0 }}
      transition={{ duration: 0.6, delay }}
```

```
    onMouseEnter={onEnter}
    onMouseLeave={onLeave}
    style={{ cursor: "pointer" }}
  >
    {/* Left wall */}
    <polygon
      points={leftFace}
      fill={color + wallAlpha}
      stroke={color + "60"}
      strokeWidth="0.8"
      style={{ transition: "all 0.3s" }}
    />
    {/* Right wall */}
    <polygon
      points={rightFace}
      fill={color + (isActive ? "66" : "18")}
      stroke={color + "60"}
      strokeWidth="0.8"
      style={{ transition: "all 0.3s" }}
    />
    {/* Top face */}
    <polygon
      points={topFace}
      fill={color + alpha}
      stroke={isActive ? color : color + "80"}
      strokeWidth={isActive ? "1.5" : "0.8"}
      style={{ transition: "all 0.3s" }}
    />
    {/* Glow on active */}
    {isActive && (
      <polygon
        points={topFace}
        fill="none"
        stroke={color}
        strokeWidth="2"
        opacity="0.4"
        filter="url(#glow)"
      />
    )}
    {/* Icon */}
    <text
      x={cx}
      y={cy - 2}
      textAnchor="middle"
      fontSize="14"
      style={{ userSelect: "none" }}
    >
```

```
        {icon}
      </text>
      {/* Label */}
      <text
        x={cx}
        y={cy + 12}
        textAnchor="middle"
        fill={isActive ? "#fff" : color}
        fontSize="7"
        fontFamily="Montserrat, sans-serif"
        fontWeight="700"
        style={{ transition: "all 0.3s", userSelect: "none" }}
      >
        {label}
      </text>
    </motion.g>
  );
}

export default function MapSchemeSection() {
  const ref = useRef(null);
  const inView = useInView(ref, { once: true, margin: "-100px" });
  const [activeZone, setActiveZone] = useState<number | null>(null);

  const active = zones.find(z => z.id === activeZone);

  // Corridor path points (isometric)
  const corridorPoints = () => {
    // Main vertical corridor at col=2.5
    const top = toIso(2.5, 0.5);
    const mid = toIso(2.5, 2.5);
    const bot = toIso(2.5, 5.5);
    // Horizontal branch at row=2.5
    const left = toIso(0, 2.5);
    const right = toIso(7, 2.5);
    return { top, mid, bot, left, right };
  };

  const c = corridorPoints();

  // Lift & travulator positions
  const lift = toIso(2.5, 5.2);
  const trav = toIso(2.5, 2.5);

  // SVG viewBox: figure out bounds
  // Grid spans col 0-7, row 0-6 roughly
  const minX = toIso(0, 6).x - 20;
```

```
const maxX = toIso(7, 0).x + 20;
const minY = toIso(0, 0).y - WALL_H - 20;
const maxY = toIso(7, 6).y + WALL_H + 40;
const vbW = maxX - minX;
const vbH = maxY - minY;

return (
  <section id="scheme" ref={ref} className="relative py-24 bg-[#0A0A0A] overflo
    <div
      className="absolute top-0 right-0 text-[20rem] leading-none text-white/[0
      style={{ fontFamily: "'Bebas Neue', sans-serif" }}
    >
      02
    </div>

    <div className="max-w-[1280px] mx-auto px-6">
      <motion.div
        initial={{ opacity: 0, x: -20 }}
        animate={inView ? { opacity: 1, x: 0 } : {}}
        transition={{ duration: 0.5 }}
        className="flex items-center gap-3 mb-4"
      >
        <div className="w-8 h-px bg-[#FF4500]" />
        <span
          className="text-[#FF4500] text-xs font-semibold tracking-[0.3em] uppe
          style={{ fontFamily: "'Montserrat', sans-serif" }}
        >
          Схема комплекса
        </span>
      </motion.div>

      <motion.h2
        initial={{ opacity: 0, y: 20 }}
        animate={inView ? { opacity: 1, y: 0 } : {}}
        transition={{ duration: 0.6, delay: 0.1 }}
        style={{ fontFamily: "'Bebas Neue', sans-serif", letterSpacing: "0.04em
        className="text-white text-[clamp(2rem,4vw,3.5rem)] leading-none mb-12"
      >
        Карта пространства<br />
        <span className="text-[#FF4500]">-3 уровень</span>
      </motion.h2>

      <div className="grid lg:grid-cols-3 gap-8 items-center">
        {/* Isometric SVG */}
        <motion.div
          initial={{ opacity: 0, scale: 0.95 }}
          animate={inView ? { opacity: 1, scale: 1 } : {}}
```

```
  transition={{ duration: 0.8, delay: 0.2 }}
  className="lg:col-span-2"
>
  <div
    className="relative bg-[#111] border border-white/10 p-4"
    style={{ clipPath: "polygon(0 0, calc(100% - 20px) 0, 100% 20px, 10
  >
    <div className="flex items-center justify-between mb-3">
      <span className="text-[#FF4500] text-xs tracking-widest uppercase
        ТЦ «Облака» · Этаж −3
      </span>
      <span className="text-white/20 text-xs" style={{ fontFamily: "'Mo
        17.000 м²
      </span>
    </div>

    <svg
      viewBox={`${minX} ${minY} ${vbW} ${vbH}`}
      className="w-full"
      style={{ aspectRatio: "16/9" }}
    >
      <defs>
        <filter id="glow" x="-50%" y="-50%" width="200%" height="200%">
          <feGaussianBlur stdDeviation="3" result="blur" />
          <feMerge>
            <feMergeNode in="blur" />
            <feMergeNode in="SourceGraphic" />
          </feMerge>
        </filter>
        {/* Floor grid */}
        <pattern id="grid" width={TILE_W} height={TILE_H} patternUnits=
          patternTransform={`rotate(-30) skewX(30)`}>
          <path d={`M ${TILE_W} 0 L 0 0 0 ${TILE_H}`} fill="none" strok
        </pattern>
      </defs>

      {/* Floor base */}
      {(() => {
        const p0 = toIso(0, 0);
        const p1 = toIso(7, 0);
        const p2 = toIso(7, 6);
        const p3 = toIso(0, 6);
        return (
          <polygon
            points={`${p0.x},${p0.y} ${p1.x},${p1.y} ${p2.x},${p2.y} ${
            fill="#0D0D0D"
            stroke="rgba(255,255,255,0.06)"
```

```
      strokeWidth="0.5"
    />
  );
})()}

{/* Isometric grid lines */}
{[0,1,2,3,4,5,6,7].map(col => {
  const a = toIso(col, 0); const b = toIso(col, 6);
  return <line key={`c${col}`} x1={a.x} y1={a.y} x2={b.x} y2={b.y
})}
{[0,1,2,3,4,5,6].map(row => {
  const a = toIso(0, row); const b = toIso(7, row);
  return <line key={`r${row}`} x1={a.x} y1={a.y} x2={b.x} y2={b.y
})}

{/* Corridor — vertical */}
{(() => {
  const tl = toIso(2.3, 0.5); const tr = toIso(2.7, 0.5);
  const bl = toIso(2.3, 5.5); const br = toIso(2.7, 5.5);
  return (
    <polygon
      points={`${tl.x},${tl.y} ${tr.x},${tr.y} ${br.x},${br.y} ${
      fill="rgba(255,69,0,0.06)"
      stroke="rgba(255,69,0,0.25)"
      strokeWidth="0.5"
      strokeDasharray="2,2"
    />
  );
})()}

{/* Corridor — horizontal */}
{(() => {
  const tl = toIso(0, 2.3); const tr = toIso(7, 2.3);
  const bl = toIso(0, 2.7); const br = toIso(7, 2.7);
  return (
    <polygon
      points={`${tl.x},${tl.y} ${tr.x},${tr.y} ${br.x},${br.y} ${
      fill="rgba(255,69,0,0.06)"
      stroke="rgba(255,69,0,0.25)"
      strokeWidth="0.5"
      strokeDasharray="2,2"
    />
  );
})()}

{/* Zones — sorted back to front for correct iso rendering */}
{[...zones].sort((a, b) => (a.col + a.row) - (b.col + b.row)).map
```

```
        <IsoBlock
          key={zone.id}
          col={zone.col} row={zone.row}
          cols={zone.cols} rows={zone.rows}
          color={zone.color}
          label={zone.label}
          icon={zone.icon}
          isActive={activeZone === zone.id}
          onEnter={() => setActiveZone(zone.id)}
          onLeave={() => setActiveZone(null)}
          delay={0.3 + i * 0.1}
        />
      ))}

      {/* Lift marker */}
      {(() => {
        const p = toIso(2.5, 5.5);
        return (
          <g>
            <circle cx={p.x} cy={p.y + 8} r="6" fill="#FF4500" opacity=
            <text x={p.x} y={p.y + 11} textAnchor="middle" fill="#FF450
            <text x={p.x} y={p.y + 20} textAnchor="middle" fill="#FF450
            {/* Arrow up */}
            <line x1={p.x} y1={p.y + 2} x2={p.x} y2={p.y - 10} stroke="
          </g>
        );
      })()}

      {/* Travulator marker */}
      {(() => {
        const p = toIso(2.5, 2.5);
        return (
          <g>
            <circle cx={p.x} cy={p.y} r="5" fill="#F39C12" opacity="0.1
            <text x={p.x} y={p.y + 3} textAnchor="middle" fill="#F39C12
            <text x={p.x + 12} y={p.y + 2} textAnchor="middle" fill="#F
          </g>
        );
      })()}

      <defs>
        <marker id="arrowUp" markerWidth="4" markerHeight="4" refX="2"
          <path d="M0,4 L2,0 L4,4 z" fill="#FF4500" />
        </marker>
      </defs>
    </svg>
  </div>
```

```
</motion.div>

{/* Right panel */}
<motion.div
  initial={{ opacity: 0, x: 20 }}
  animate={inView ? { opacity: 1, x: 0 } : {}}
  transition={{ duration: 0.7, delay: 0.4 }}
  className="flex flex-col gap-3"
>
  {/* Active zone info */}
  {active ? (
    <motion.div
      key={active.id}
      initial={{ opacity: 0, y: 10 }}
      animate={{ opacity: 1, y: 0 }}
      className="p-4 border mb-2"
      style={{
        borderColor: active.color + "60",
        background: active.color + "10",
        clipPath: "polygon(0 0, calc(100% - 12px) 0, 100% 12px, 100% 10
      }}
    >
      <div className="text-3xl mb-2">{active.icon}</div>
      <h3
        className="text-white text-xl mb-1"
        style={{ fontFamily: "'Bebas Neue', sans-serif", letterSpacing:
      >
        {active.label}
      </h3>
      <p className="text-white/50 text-xs leading-relaxed" style={{ fon
        {active.desc}
      </p>
    </motion.div>
  ) : (
    <div className="p-4 border border-white/5 mb-2" style={{ clipPath:
      <p className="text-white/20 text-xs" style={{ fontFamily: "'Monts
        Наведи на зону чтобы узнать подробнее
      </p>
    </div>
  )}

  <h3
    className="text-white/40 text-xs uppercase tracking-widest"
    style={{ fontFamily: "'Montserrat', sans-serif" }}
  >
    Зоны комплекса
  </h3>
```

```
            {zones.map((zone) => (
              <div
                key={zone.id}
                onMouseEnter={() => setActiveZone(zone.id)}
                onMouseLeave={() => setActiveZone(null)}
                className="flex items-center gap-3 p-3 border transition-all dura
                style={{
                  borderColor: activeZone === zone.id ? zone.color + "60" : "rgba
                  background: activeZone === zone.id ? zone.color + "10" : "rgba(
                  clipPath: "polygon(0 0, calc(100% - 8px) 0, 100% 8px, 100% 100%
                }}
              >
                <div className="w-3 h-3 flex-shrink-0 rounded-sm" style={{ backgr
                <span className="text-sm" style={{ fontFamily: "'Montserrat', san
                  {zone.icon} {zone.label}
                </span>
              </div>
            ))}

            {/* Infrastructure */}
            <div className="mt-2 flex flex-col gap-2">
              <h3 className="text-white/40 text-xs uppercase tracking-widest" sty
                Инфраструктура
              </h3>
              <div className="flex items-center gap-2 text-xs text-white/40" styl
                <span>🛗</span><span>Лифт</span>
              </div>
              <div className="flex items-center gap-2 text-xs text-white/40" styl
                <span>⚡</span><span>Траволатор</span>
              </div>
            </div>
          </motion.div>
        </div>
      </div>
    </section>
  );
}
```